

复旦大学

面向对象程序设计

定义新类型：类

周雅倩
zhouyagqian@fudan.edu.cn
 2026/4/15

1

面向对象程序设计

定义新类型

- 内置类型：核心语言的一部分
 - char , int, double
- 类
 - 库中的类：string, vector, istream
 - 程序员设计的类
- C++程序设计思想：允许程序员创建与内置类型一样易用的类型。

2026/4/15

2

面向对象程序设计

回顾Student_info

```
struct Student_info{
    std::string name;
    double midterm, final;
    std::vector<double> homework;
};
```

- 缺点：不好用
 - 若数据读入失败，则成员值无效。导致产生不正确的结果或程序崩溃。
 - 若要检测对象是否包含有效数据，需要用户了解实现细节。
- 无法支持一些约定
 - 如：一个学生信息一旦从文件读入，则不会改变
- “接口”分散
 - 可以改进，但费事：把改变对象的状态的函数（如read）放在一个单独的头文件中

2026/4/15

3

面向对象程序设计

Student_info的使用

- 输入


```
read(cin, record)
```
- 输出


```
cout << students[i].name
      << string(maxlen + 1 - students[i].name.size(), ' ');
      double final_grade = grade(students[i]);
```
- 排序


```
sort(students.begin(),students.end(),compare);
```

2026/4/15

4

面向对象程序设计

类

- 使用student_info结构类型的程序员都可以（也必须）直接控制这些数据元素。
- 但是，我们希望隐藏其存储方式的实现细节，而不是允许用户直接访问这些数据。
- 特别地，希望用户只能通过函数来访问这些对象。这些函数就是类的接口。

2026/4/15

5

面向对象程序设计

第一个成员函数

```
struct Student_info{
    std::string name;
    double midterm, final;
    std::vector<double> homework;
    std::istream & read(std::istream);
};

istream & Student_info::read(std::istream &in)
{
    in >> name >> midterm >> final;
    read_hw(in, homework);
    return in;
}
```

调用：
`s.read(cin);`

istream & read(istream & is, Student_info &s)

```
{
    is >> s.name >> s.midterm >> s.final;
    read_hw(is, s.homework);
    return is;
}
```

- 与原先代码的不同之处
 - 函数的名字是 Student_info::read而不是简单的read
 - 由于它是成员函数，所以不需要把一个Student_info对象作为参数传递，而且在函数体中不需要定义一个Student_info对象。
 - 可以直接访问对象的数据元素。

2026/4/15

6

面向对象程序设计

const函数

```
double Student_info::grade() const{
    return ::grade(midterm, final, homework);
}

double grade(const Student_info &s){
    return grade(s.midterm, s.final, s.homework);
}
```

- 紧跟在grade函数的参数列表后的const关键字，表示该函数不能改变它所操作的对象的数据。
- 把::放在一个名字前表示我们要使用这个名字的一个版本，但是这个名字决不是任何对象的成员

2026/4/15

7

面向对象程序设计

const 函数

- 通过在参数列表后插入const关键字，可以把成员函数定义为const函数
 - double Student_info::grade() const
- 这样的成员函数不能改变调用它们的对象的状态
- const对象只能调用const成员函数

2026/4/15

8

面向对象程序设计

成员函数

- 成员函数是通过特定的对象来隐式地调用。
- 在一个成员函数中，引用到的成员数据和函数会隐式的绑定到调用这个函数的对象上。
- 在类外定义成员函数：
 - class-name::member-name

2026/4/15

9

面向对象程序设计

保护

<pre>class Student_info{ public: //接口 double grade() const; std::istream & read(std::istream &); private: std::string name; double midterm, final; std::vector<double> homework; };</pre>	<pre>struct Student_info{ double grade() const; std::istream & read(std::istream &); std::string name; double midterm, final; std::vector<double> homework; };</pre>
---	--

2026/4/15

10

面向对象程序设计

保护标签

- 保护标签控制对类的成员的访问
 - public:公有成员可以被所有用户访问
 - private:私有成员只有这个类的成员可以访问，用户不能访问。
 - protected
- 在一个类中，保护标签可以按任意顺序出现，并且可以多次出现。

2026/4/15

11

面向对象程序设计

struct Vs. class

- 用户定义的类型可以是struct，也可以是class。
- 它们之间唯一的区别在于默认的方式不同
 - struct: 公有
 - class: 私有
- 一般struct用来表示简单的类型

2026/4/15

12

例子

```
class Student_info{
public:
    double grade() const;
    //etc
};

class Student_info{
    std::string name;
    //etc
public:
    double grade() const;
    //etc
};
```

```
struct Student_info{
    double grade() const;
    //etc
};

struct Student_info{
private:
    std::string name;
    //etc
public:
    double grade() const;
    //etc
};
```

2026/4/15

13

访问器函数

```
class Student_info{
public:
    //接口
    double grade() const;
    std::istream & read(std::istream
    &);
    std::string name() const{return
    n;};
private:
    std::string n;
    double midterm, final;
    std::vector<double> homework;
};
```

```
class Student_info{
public:
    //接口
    double grade() const;
    std::istream & read(std::istream
    &);
private:
    std::string name;
    double midterm, final;
    std::vector<double> homework;
};
```

象name这样的函数常常叫做访问器函数，允许访问隐藏的数据。

2026/4/15

14

内联函数

成员函数可以定义在类定义中，也可以定义在类定义外。

```
inline string Student_info::name() const{ return n; }
```

- 定义在类定义中的成员函数称为内联函数，可以避免调用函数的开销。
- 在程序编译时，编译器将程序中出现的内联函数的调用表达式用内联函数的函数体来进行替换。

2026/4/15

15

compare函数

```
bool compare(const Student_info &x, const Student_info &y)
{
    return x.name() < y.name();
}
```

注意：这里compare函数是非成员函数

2026/4/15

16

非成员函数

- 通用规则：
 - 如果这个函数会改变对象的状态，那么这个函数就应该成为这个对象的成员。
- 我们在新的设计中把read和grade函数都变成了成员函数。那么compare函数呢？
 - 考虑这个函数是做什么的？比较大小。

2026/4/15

17

检测对象是否为空

- 以下代码会产生意外：


```
Student_info s;
cout << s.grade() << endl;
```

 （因为homework为空）：
- 一种方式：直接检查homework成员
 - 缺点：破坏封装性
- 更好的方式：提供检测函数


```
class Student_info{
public:
    bool valid()const {return homework.empty();}
    //etc
}
```

2026/4/15

18

面向对象程序设计

封装性

- 封装就是将接口和实现细节分离开来的过程。
- 一般来说接口是不会轻易改变的，但是实现细节可能由于各种原因发生变化，
- 封装就起到了在变与不变之间划界线的作用。

抽象性 - 封装性 - 继承性 - 多态性

2026/4/15

19

面向对象程序设计

Student_info类

```
class Student_info{
public:
    double grade() const;
    std::istream & read(std::istream &);
    bool valid()const {return homework.empty();}
    std::string name() const{return n;}
private:
    std::string n;
    double midterm, final;
    std::vector<double> homework;
};
bool compare(const Student_info &x, const Student_info &y);
```

2026/4/15

20

面向对象程序设计

构造函数

- 是特殊的成员函数，定义了对象如何初始化。
- 名字和类的名字相同，没有返回值。
- 一个类可以定义多个构造函数，只要参数个数或者类型不同。

```
class Student_info{
public:
    Student_info();           定义对象: Student_info s;
    Student_info(std::istream&); Student_info s(cin);
};
```

2026/4/15

21

面向对象程序设计

构造函数

- 不能显式的调用一个构造函数，但是创建类的一个对象时就会自动的调用适当的构造函数
- 如果没有定义任何构造函数编译器就会为我们合成一个。
- 名字和类的名字相同，没有返回值。
- 一个类可以定义多个构造函数，只要参数个数或者类型不同。
- 编程习惯：
 - 从任何一个构造函数退出前，确保每个数据成员都有一个有意义的值。

2026/4/15

22

面向对象程序设计

默认构造函数

- 不带有参数的构造函数被认为是默认构造函数。它的工作是确保对象的数据成员被适当地初始化。
- Student_info: 初始化表示还没有读取一条记录
 - homework为空
 - n为空string
 - midterm和final为0

```
Student_info::Student_info() : midterm(0), final(0){}
```

2026/4/15

23

面向对象程序设计

构造函数初始化列表

```
Student_info::Student_info() : midterm(0), final(0){}
```

- 构造函数初始化列表是一个用逗号分隔的列表，这个列表包含的是 member-name(value)对。没有明显的初始化的数据成员会被隐式的初始化。
- 通常来说，显式地为一个成员提供一个初始值，要比在构造函数的函数体中对这个成员进行赋值要好得多，可以避免做两次相同的工作。

2026/4/15

24

对象创建过程

- 当我们创建一个新的类对象时，会发生下面几个连续的步骤
 - 系统会分配内存空间，以保存这个对象
 - 根据构造函数的初始化列表，初始化这个对象
 - 执行构造函数的函数体

2026/4/15

25

对象初始化规则

- 若一个类定义了一个或多个构造函数，那么适当的构造函数会完全控制这个类的对象的初始化
- 若一个对象的成员是内置类型的话，那么值初始化会把它设置为0，而默认初始化会提供一个不明确的值。
- 若一个对象的类没有定义任何构造函数，那么对这个对象进行值初始化或默认初始化就会对它的每个数据成员进行，整个过程递归进行。

2026/4/15

26

初始化顺序

- 成员初始化的顺序取决于它们在类中声明的顺序。
- 所以在用类的一个数据成员来初始化另外一个成员时一定要小心。
- 安全的做法：在构造函数的函数体中对这些成员赋值，而不是在构造函数初始化列表中初始化它们。
 - 因为成员初始化的顺序取决于它们在类中声明的顺序

2026/4/15

27

带有参数的构造函数

```
Student_info::Student_info(istream & is){
    read(is);
}
```

2026/4/15

28

使用Student_info类

```
int main()
{
    vector<Student_info> students;
    Student_info record;
    string::size_type maxlen = 0;
    while(record.get() < 0)
    {
        if(record.name().size() > maxlen) maxlen = record.name().size();
        students.push_back(record);
    }
    sort(students.begin(), students.end(), compare);
    for(std::vector<Student_info>::iterator iter = students.begin(); iter != students.end(); iter++)
    {
        cout << iter->name()
              << string(maxlen + 1 - iter->name().size(), ' ');
        try
        {
            double final_grade = iter->grade();
            streamsize prec = cout.precision();
            cout << "Your final grade is " << setprecision(3)
                  << final_grade
                  << setprecision(prec) << endl;
        }
        catch(domain_error e)
        {
            cout << e.what();
        }
        cout << endl;
    }
    return 0;
}
```

2026/4/15

29

小结

- 用户自定义类型
 - class
- 保护标识符
 - public, private
- 成员函数
 - 调用时对象绑定，const成员函数
- 构造函数
 - 默认构造函数，系统合成构造函数
 - 构造函数初始化程序列表，对象初始化过程

2026/4/15

30